
Homework 1: Final Submission

IE 598: Reinforcement Learning and Learning-Based Control

Dynamic programming, partial observation, and convex interpolation

Prachit Gupta

NetID: prachit2

September 18, 2026

Contents

1	Question 1: Inventory Control	1
1.1	Terminal stage	1
1.2	Stage $t = 1$	1
1.3	Stage $t = 0$	2
2	Question 2: GridWorld	3
2.1	1(a): Finite-horizon dynamic programming	3
2.2	1(b): Heat maps of J_0^* and J_6^*	3
2.3	1(c): Comparing J_0^* and J_6^*	4
2.4	2(a): Exact Bayesian belief update	4
2.5	2(b): Belief-state heat maps	5
2.6	3: MAP policy	6
2.7	4: QMDP policy	6
2.8	5: Policy performance	6
3	Question 3: Discretization of a Convex Space	7
3.1	(a) Prove $\hat{J}_k(x) \leq \tilde{J}_k(x)$	7
3.2	(b) Prove $J_{N-1}(x) = \hat{J}_{N-1}(x) \leq \tilde{J}_{N-1}(x)$	7
3.3	(c) Prove $J_{N-2}(x) \leq \hat{J}_{N-2}(x) \leq \tilde{J}_{N-2}(x)$	8
3.4	(d) Optional bonus: proof for every stage	8

1 Question 1: Inventory Control

The disturbance and one-period inventory cost are

$$w_t = \begin{cases} 1, & \text{with probability } 1/2, \\ 3, & \text{with probability } 1/2, \end{cases} \quad c(x) = \begin{cases} x, & x \geq 0, \\ -1000x, & x < 0. \end{cases}$$

The stage and terminal costs are

$$g_t(m_t, u_t) = c(m_t) + 2u_t, \quad g_2(m_2) = c(m_2).$$

1.1 Terminal stage

At $t = N = 2$,

$$J_2^*(m_2) = g_2(m_2) = c(m_2) = \begin{cases} m_2, & m_2 \geq 0, \\ -1000m_2, & m_2 < 0. \end{cases}$$

1.2 Stage $t = 1$

Backward dynamic programming gives

$$\begin{aligned} J_1^*(m_1) &= \min_{u_1} \mathbb{E}_{w_1} [c(m_1) + 2u_1 + J_2^*(m_1 + u_1 - w_1)] \\ &= c(m_1) + \min_{u_1} F_1(u_1), \end{aligned}$$

where

$$\begin{aligned} F_1(u_1) &= 2u_1 + \frac{1}{2}J_2^*(m_1 + u_1 - 1) + \frac{1}{2}J_2^*(m_1 + u_1 - 3) \\ &= \begin{cases} -998u_1 - 1000m_1 + 2000, & m_1 + u_1 < 1, \\ \frac{-995u_1 - 999m_1 + 2999}{2}, & 1 \leq m_1 + u_1 < 3, \\ 3u_1 + m_1 - 2, & m_1 + u_1 \geq 3. \end{cases} \end{aligned}$$

The slopes of the three pieces are -998 , $-995/2$, and 3 . Therefore, the minimum occurs at the second breakpoint:

$$u_1^* = 3 - m_1.$$

Substitution gives

$$F_1(u_1^*) = 3(3 - m_1) + m_1 - 2 = 7 - 2m_1,$$

and hence

$$J_1^*(m_1) = c(m_1) + 7 - 2m_1, \quad \mu_1^*(m_1) = 3 - m_1.$$

Equivalently,

$$J_1^*(m) = \begin{cases} 7 - 1002m, & m < 0, \\ 7 - m, & m \geq 0. \end{cases}$$

1.3 Stage $t = 0$

Continue the same recursion:

$$\begin{aligned} J_0^*(m_0) &= \min_{u_0} \mathbb{E}_{w_0} [c(m_0) + 2u_0 + J_1^*(m_0 + u_0 - w_0)] \\ &= c(m_0) + \min_{u_0} F_0(u_0), \end{aligned}$$

where

$$\begin{aligned} F_0(u_0) &= 2u_0 + \frac{1}{2}J_1^*(m_0 + u_0 - 1) + \frac{1}{2}J_1^*(m_0 + u_0 - 3) \\ &= \begin{cases} -1000u_0 - 1002m_0 + 2011, & m_0 + u_0 < 1, \\ \frac{-999u_0 - 1003m_0 + 3021}{2}, & 1 \leq m_0 + u_0 < 3, \\ u_0 - m_0 + 9, & m_0 + u_0 \geq 3. \end{cases} \end{aligned}$$

The slopes are -1000 , $-999/2$, and 1 , so the minimum again occurs at the second breakpoint:

$$u_0^* = 3 - m_0.$$

At this control,

$$F_0(u_0^*) = (3 - m_0) - m_0 + 9 = 12 - 2m_0.$$

Therefore,

$$J_0^*(m_0) = c(m_0) + 12 - 2m_0, \quad \mu_0^*(m_0) = 3 - m_0.$$

In particular,

$$J_0^*(0) = c(0) + 12 = 12.$$

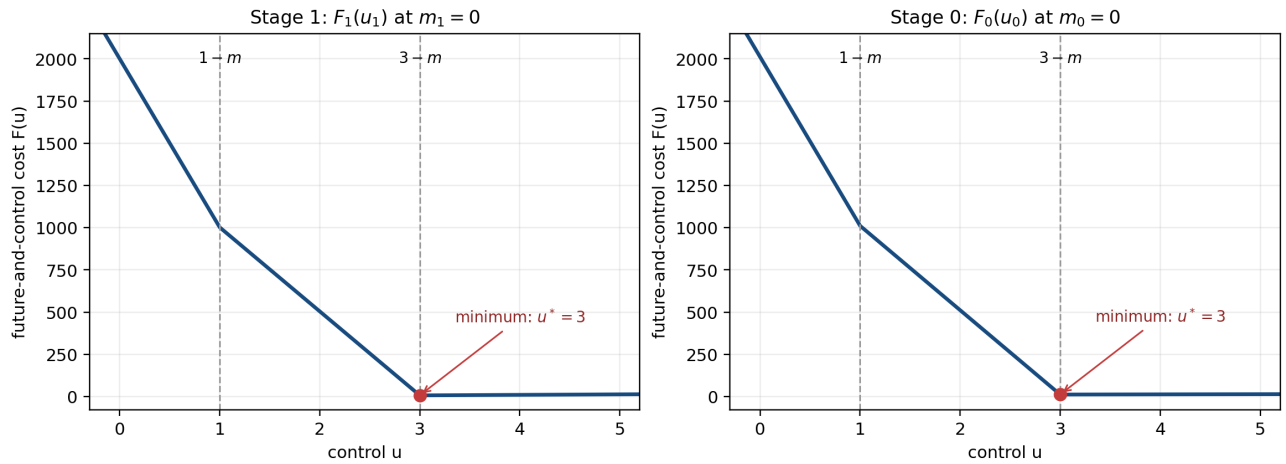


Figure 1: Piecewise-linear functions minimized at stages 1 and 0, shown for $m_1 = m_0 = 0$. The dashed lines mark the two breakpoints. The same geometry shifts horizontally by $-m_t$ for a general inventory state.

2 Question 2: GridWorld

The implementation uses the given 6×6 grid, three obstacle cells, absorbing goal $G = (5, 5)$, horizon $N = 15$, transition probabilities in `BasicFunctions.py`, and the supplied random seed 24. Only `MDP.py`, `POMDP.py`, and `MAP.py` were edited.

2.1 1(a): Finite-horizon dynamic programming

The terminal values are initialized as $J_N^*(x) = c_N(x)$. For $t = N - 1, \dots, 0$, the code evaluates

$$J_t^*(x) = \min_{u \in \mathcal{U}} \left[c(x, u) + \sum_{x'} P(x' | x, u) J_{t+1}^*(x') \right].$$

The completed portion of `MDP.py` is:

```

1 J_opt[N, :] = bf.terminal_costs
2
3 for t in range(N - 1, -1, -1):
4     for s_index in range(S):
5         J_opt[t, s_index] = min([
6             stage_cost_c(s_index, a)
7             + sum([
8                 transition_prob[a, s_index, s_next]
9                 * J_opt[t + 1, s_next]
10                for s_next in range(S)
11            ])
12         for a in range(A)
13     ])
```

2.2 1(b): Heat maps of J_0^* and J_6^*

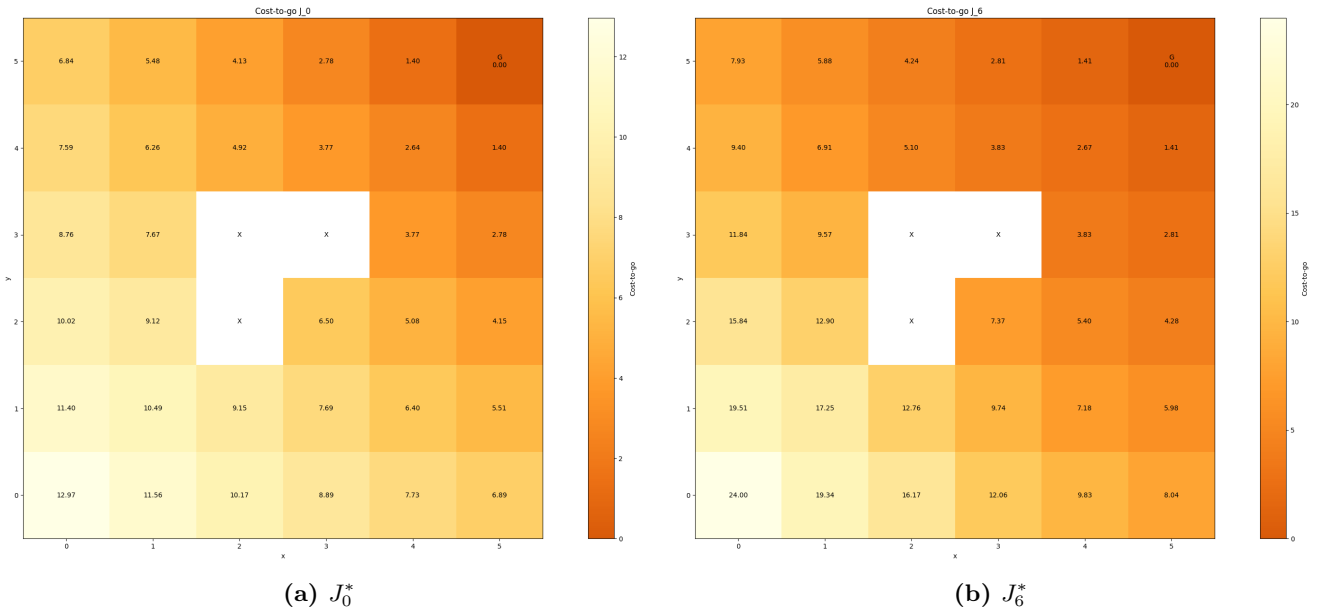


Figure 2: Optimal finite-horizon cost-to-go heat maps. White cells marked X are obstacles and G is the absorbing goal.

2.3 1(c): Comparing J_0^* and J_6^*

As evident in the heat maps, for the same state x , $J_6^*(x)$ is generally larger than $J_0^*(x)$. At $t = 6$, the robot has fewer remaining time steps before the terminal horizon, so it has fewer opportunities to reach the goal. Hence, it has a higher probability of remaining outside the goal at $t = N$ and incurring the terminal penalty of 15. In contrast, starting from $t = 0$ gives the robot more time to reach the absorbing goal, after which the stage cost is zero.

Although the Bellman equation adds the current stage cost to the expected future cost, this does not imply that the cost-to-go must increase as we move backward in time. The future-cost term also changes because an earlier starting time provides more opportunities to reach the goal and avoid the relatively large terminal penalty. In this case, that benefit outweighs the extra stage costs, so $J_0^*(x)$ is generally smaller than $J_6^*(x)$.

2.4 2(a): Exact Bayesian belief update

For an action u_t and new observation y_{t+1} , define

$$\text{Term1}(x') = P(y_{t+1} \mid x') \sum_x P(x' \mid x, u_t) b_t(x).$$

Then

$$b_{t+1}(x') = \frac{\text{Term1}(x')}{\sum_{\tilde{x}} \text{Term1}(\tilde{x})}.$$

The completed portion of `POMDP.py` is:

```

1 term1 = np.zeros(S)
2
3 for s_next in range(S):
4     for s in range(S):
5         term1[s_next] += (
6             belief[s]
7             * transition_prob[action_index, s, s_next]
8             * observation_prob[s_next, observation_index]
9         )
10
11 updated_belief = np.zeros(S)
12 if term1.sum() > 0:
13     updated_belief = term1 / term1.sum()
14
15 return updated_belief

```

2.5 2(b): Belief-state heat maps

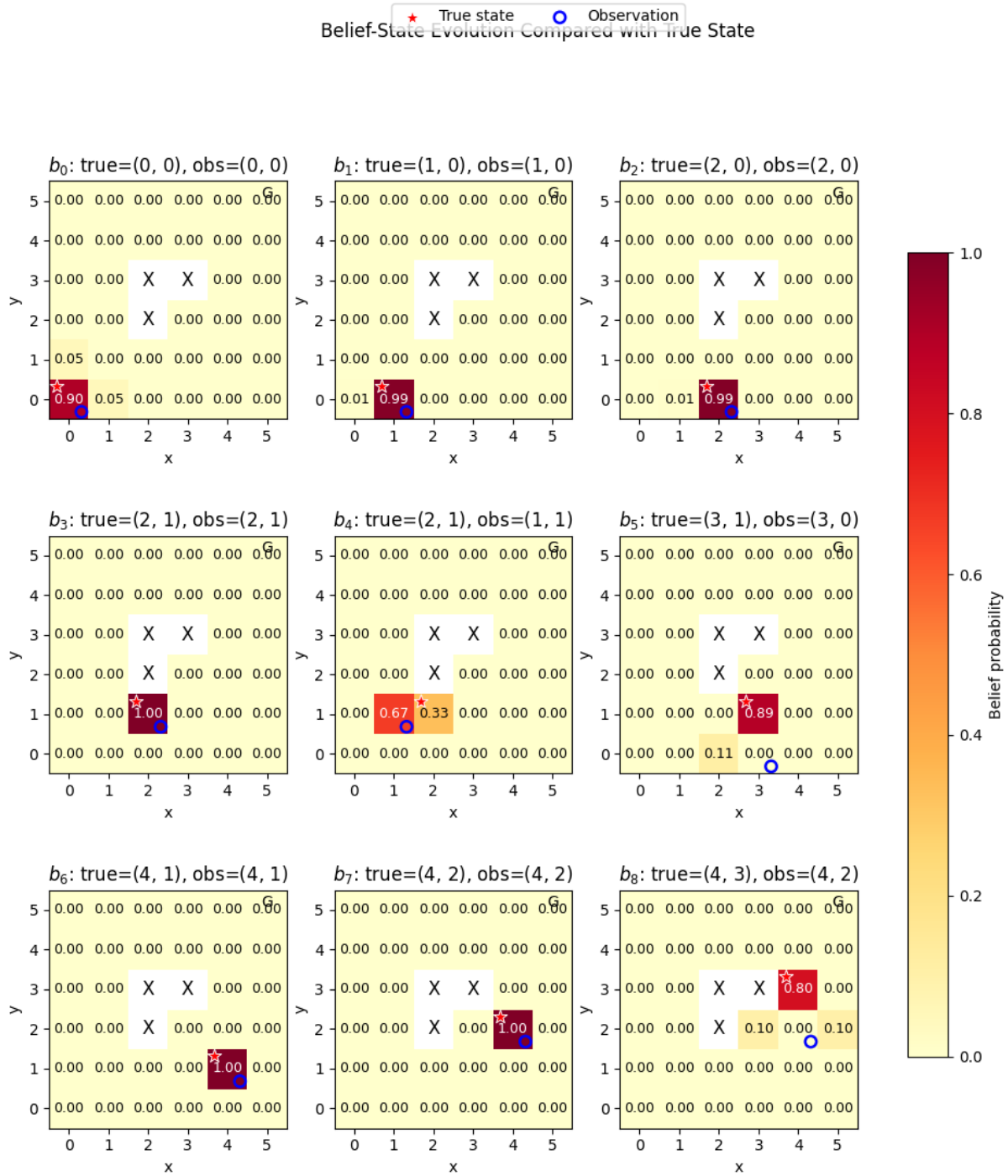


Figure 3: Belief-state evolution $b_0, \dots, b_8$. The red star is the true state and the blue circle is the noisy observation. A common color scale is used across all nine panels.

The belief assigned to the true state at times 0 through 8 is, respectively,

$$0.9000, 0.9856, 0.9918, 0.9995, 0.3333, 0.8889, 1.0000, 1.0000, 0.8000.$$

The reductions at $t = 4$, $t = 5$, and $t = 8$ correspond to deliberately noisy observations.

2.6 3: MAP policy

The MAP estimate is

$$\hat{x}_t = \arg \max_x b_t(x).$$

The action treats $\hat{x}_t$ as the true state and minimizes its one-step MDP cost. The completed portion of `MAP.py` is:

```

1 most_likely_state = np.argmax(belief)
2
3 best_action = np.argmin([
4     stage_cost_c(most_likely_state, a)
5     + sum([
6         transition_prob[a, most_likely_state, s_next]
7         * J_opt[t + 1, s_next]
8         for s_next in range(S)
9     ])
10    for a in range(A)
11 ])
12
13 return best_action

```

2.7 4: QMDP policy

The QMDP policy uses the entire belief distribution rather than only the most likely state. For each action, it computes

$$\sum_x b_t(x) \left[c(x, u) + \sum_{x'} P(x' | x, u) J_{t+1}^*(x') \right]$$

and chooses the action with the smallest belief-weighted expected cost. Intuitively, QMDP accounts for uncertainty in the robot's location instead of committing to a single estimated state.

2.8 5: Policy performance

Using 1,000 episodes per policy, start state (0,0), and the provided seed 24 gives:

Table 1: MAP and QMDP performance under identical simulation settings.

Policy	Success rate	Average cumulative cost	Avg. time to goal
MAP	0.925	13.085	11.714
QMDP	0.941	12.774	11.694

QMDP performs better in all three reported metrics in this experiment: it has a higher success rate, a lower average cumulative cost, and a slightly lower average time to the goal among successful runs.

The reason is that QMDP accounts for uncertainty in the current state rather than assuming that the most likely state is the only possible state. MAP can choose an action that is advantageous at its single most likely state but poor if the robot is actually in another state. This matters especially when the second most likely state has probability close to that of the MAP state.

One might initially expect MAP to reach the goal faster because it optimizes only for the most likely state. The results do not show that tradeoff here: QMDP's average successful-run time is smaller by 0.020 steps. That difference is very small, so the experiment does not support a general claim that QMDP is inherently faster. It does show that QMDP is better on all three requested measures for this fixed setup and seed.

3 Question 3: Discretization of a Convex Space

Let $S = \text{conv}\{s_1, \dots, s_M\}$ and suppose $\hat{J}_k$ is real-valued and convex over S . Recall that

$$\tilde{J}_k(x) = \min \left\{ \sum_{i=1}^M \lambda_i \hat{J}_k(s_i) \mid \sum_{i=1}^M \lambda_i s_i = x, \sum_{i=1}^M \lambda_i = 1, \lambda_i \geq 0 \right\},$$

and

$$\hat{J}_k(x) = \min_{u \in U_k(x)} \mathbb{E} \left[g_k(x, u, w_k) + \tilde{J}_{k+1}(f_k(x, u, w_k)) \right].$$

Let μ_k be a policy attaining this latter minimum, and let J_k be the actual cost-to-go when the policy sequence $\vec{\mu}$ is used.

3.1 (a) Prove $\hat{J}_k(x) \leq \tilde{J}_k(x)$

Fix $x \in S$. Because S is the convex hull of its finite subset $\hat{S} = \{s_1, \dots, s_M\}$, there is at least one feasible vector λ such that

$$x = \sum_{i=1}^M \lambda_i s_i, \text{quad} \sum_{i=1}^M \lambda_i = 1, \text{quad} \lambda_i \geq 0.$$

By the assumed convexity of $\hat{J}_k$ and Jensen's inequality,

$$\hat{J}_k(x) = \hat{J}_k \left(\sum_{i=1}^M \lambda_i s_i \right) \leq \sum_{i=1}^M \lambda_i \hat{J}_k(s_i).$$

This holds for every feasible representation λ of x , including a minimizing representation. Taking the minimum of the right-hand side over all feasible λ yields

$$\hat{J}_k(x) \leq \tilde{J}_k(x), \quad x \in S, \quad k = 0, 1, \dots, N-1.$$

3.2 (b) Prove $J_{N-1}(x) = \hat{J}_{N-1}(x) \leq \tilde{J}_{N-1}(x)$

At the terminal stage,

$$J_N(x) = \tilde{J}_N(x) = \hat{J}_N(x) = g_N(x).$$

Let $\bar{u} = \mu_{N-1}(x)$ denote the action selected by the one-step lookahead policy. From the definition of the actual policy cost-to-go,

$$\begin{aligned} J_{N-1}(x) &= \mathbb{E} \left[g_{N-1}(x, \bar{u}, w_{N-1}) + J_N(f_{N-1}(x, \bar{u}, w_{N-1})) \right] \\ &= \mathbb{E} \left[g_{N-1}(x, \bar{u}, w_{N-1}) + \tilde{J}_N(f_{N-1}(x, \bar{u}, w_{N-1})) \right]. \end{aligned}$$

Because $\bar{u} = \mu_{N-1}(x)$ minimizes the last expression, its value is exactly the definition of $\hat{J}_{N-1}(x)$. Thus

$$J_{N-1}(x) = \hat{J}_{N-1}(x).$$

Part (a) gives $\hat{J}_{N-1}(x) \leq \tilde{J}_{N-1}(x)$. Therefore,

$$J_{N-1}(x) = \hat{J}_{N-1}(x) \leq \tilde{J}_{N-1}(x), \quad x \in S.$$

3.3 (c) Prove $J_{N-2}(x) \leq \hat{J}_{N-2}(x) \leq \tilde{J}_{N-2}(x)$

Let $\bar{u} = \mu_{N-2}(x)$. By the definition of the actual cost-to-go,

$$J_{N-2}(x) = \mathbb{E}[g_{N-2}(x, \bar{u}, w_{N-2}) + J_{N-1}(f_{N-2}(x, \bar{u}, w_{N-2}))].$$

Part (b) states that $J_{N-1}(z) \leq \tilde{J}_{N-1}(z)$ for every $z \in S$. Applying this pointwise at the random next state and then taking expectations gives

$$\begin{aligned} J_{N-2}(x) &\leq \mathbb{E}[g_{N-2}(x, \bar{u}, w_{N-2}) + \tilde{J}_{N-1}(f_{N-2}(x, \bar{u}, w_{N-2}))] \\ &= \min_{u \in U_{N-2}(x)} \mathbb{E}[g_{N-2}(x, u, w_{N-2}) + \tilde{J}_{N-1}(f_{N-2}(x, u, w_{N-2}))] \\ &= \hat{J}_{N-2}(x), \end{aligned}$$

where the equality uses the fact that $\bar{u} = \mu_{N-2}(x)$ is the minimizing action. Finally, part (a) supplies the second inequality, so

$$J_{N-2}(x) \leq \hat{J}_{N-2}(x) \leq \tilde{J}_{N-2}(x), \quad x \in S.$$

3.4 (d) Optional bonus: proof for every stage

We use backward induction. At $k = N$, the terminal functions agree:

$$J_N(x) = \hat{J}_N(x) = \tilde{J}_N(x) = g_N(x).$$

Equivalently, part (b) establishes the first nonterminal base case at $k = N - 1$.

Assume for some $k + 1 \leq N - 1$ that

$$J_{k+1}(z) \leq \hat{J}_{k+1}(z) \leq \tilde{J}_{k+1}(z) \text{ for every } z \in S.$$

Let $\bar{u} = \mu_k(x)$. Then

$$\begin{aligned} J_k(x) &= \mathbb{E}[g_k(x, \bar{u}, w_k) + J_{k+1}(f_k(x, \bar{u}, w_k))] \\ &\leq \mathbb{E}[g_k(x, \bar{u}, w_k) + \tilde{J}_{k+1}(f_k(x, \bar{u}, w_k))] \\ &= \min_{u \in U_k(x)} \mathbb{E}[g_k(x, u, w_k) + \tilde{J}_{k+1}(f_k(x, u, w_k))] \\ &= \hat{J}_k(x). \end{aligned}$$

Part (a), which follows from convexity at every stage, gives

$$\hat{J}_k(x) \leq \tilde{J}_k(x).$$

Thus the induction closes, proving

$$J_k(x) \leq \hat{J}_k(x) \leq \tilde{J}_k(x), \quad x \in S, \quad k = 0, 1, \dots, N - 1.$$